



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

MedAssist

The domain of the Project: Healthcare Information Technology & Artificial Intelligence

COURSE NAME

AI & ML

Team Mentors (and their designation):

Ms. Rajalakshmi C

Team Members:

Ms. Divya Goyal

Period of the project

20 July 2026 to 20 September 2026



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

Declaration

The project titled MedAssist has been mentored by Rajalakshmi C, organised by SURE Trust, from April 2023 to August 2023, for the benefit of the educated unemployed rural youth for gaining hands-on experience in working on industry relevant projects that would take them closer to the prospective employer. I declare that to the best of my knowledge the members of the team mentioned below, have worked on it successfully and enhanced their practical knowledge in the domain.

Team Members:

Ms. Divya Goyal

Mentor's Name

Ms. Rajalakshmi C

Designation—Company Name

Prof. Radhakumari

Executive Director & Founder

SURE Trust



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

Table of contents

1. Executive summary
2. Introduction
3. Project Objectives
4. Methodology & Results
5. Social / Industry relevance of the project
6. Learning & Reflection
7. Future Scope & Conclusion



Executive Summary

- A concise overview of the project's objectives, methods, key findings, and recommendations.
- Highlights the most important aspects of the project for executives or stakeholders.

MedAssist is an AI-powered rural health information and diagnostic assistant designed to bridge the healthcare knowledge gap in underserved communities. The project leverages Retrieval-Augmented Generation (RAG) over trusted MedlinePlus sources, combined with Large Language Models (LLM) via Groq API, to deliver accurate, structured medical information about diseases, medications, symptoms, and diagnostic tests.

The system features four core modules: (1) Disease Information Lookup with clickable diagnostic test details, (2) Drug Information Query with fuzzy brand-to-generic name matching, (3) Medical Report Analyzer using OCR and AI summarization, and (4) Symptom-based Disease Prediction with confidence scoring and urgency classification.

A key innovation is the multilingual and voice-enabled interface supporting 12 Indian languages with text-to-speech output, making the platform accessible to illiterate and non-English speaking populations. The architecture includes robust fallback mechanisms — 3-model LLM fallback chains, keyword-based test retrieval, and drug-label filtering — ensuring reliability even with intermittent connectivity.

All medical data is sourced from authoritative databases including MedlinePlus (NIH) and OpenFDA (FDA), ensuring information accuracy. The system is built using Python, Streamlit, ChromaDB vector database, and SentenceTransformers embeddings, making it lightweight and deployable in resource-constrained environments.



Introduction

- **Background and context of the project**

India's rural healthcare system faces a critical challenge: over 600 million people reside in villages with limited access to qualified medical professionals. The doctor-to-patient ratio in rural areas is approximately 1:10,000, far below the WHO recommended standard of 1:1,000. This gap leads to widespread self-medication, delayed diagnoses, and preventable health complications. While the internet contains vast medical information, it is predominantly in English, creating a language barrier for non-English speaking populations. Additionally, medical content is often too complex for laypersons to understand, and there is no centralized platform that provides symptom checking, disease information, drug details, and report analysis in one place.

- **Problem statement or goals of the project**

There is a pressing need for an accessible, reliable, and multilingual health information system that can:

- Provide instant disease information with symptoms, tests, and treatment options
- Enable drug lookup with dosage, side effects, and interaction warnings
- Analyze medical reports without requiring a doctor's immediate presence
- Predict possible diseases from described symptoms
- Deliver all information in local languages with voice output for illiterate users

- **Scope and limitations of the project**

Scope: The project covers four major healthcare information domains (disease, drug, report, symptom) with support for 12 Indian languages and voice output.

Limitations:

- The system provides informational content only and does not replace professional medical diagnosis
- LLM responses depend on API availability and may occasionally require fallback processing
- The symptom checker uses a predefined disease-symptom mapping database and may not cover rare conditions



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

- Voice output is limited to 450 characters per audio clip

- **Innovation component in the project**

MedAssist introduces several innovative features:

- **Smart Drug-Label Filtering:** Automatically filters out FDA medication guides from MedlinePlus search results to prevent drug label text from contaminating disease information
- **Fuzzy Test Matching:** Handles variations like "A1C" → "Hemoglobin A1C (HbA1c) Test" using normalized string matching
- **3-Model LLM Fallback:** Automatically tries qwen-27b → gpt-oss-120b → gpt-oss-20b if one model fails
- **Clickable Diagnostic Tests:** Disease results include clickable test buttons that link to a 1000+ test database with preparation, procedure, and normal values



Project Objectives

- Clearly defined objectives and goals of the project

Primary Objectives

1. **Develop a centralized health information platform** that provides instant, accurate disease information including symptoms, diagnostic tests, precautions, treatment, risk factors, and when to consult a doctor.
2. **Build a drug information system** that allows users to search medicines by generic name, brand name, or symptom, and returns structured data on indications, dosage, side effects, warnings, and drug interactions.
3. **Create a medical report analyzer** that can extract text from uploaded PDF or image reports using OCR, then use AI to generate a summary, identify abnormal values, and provide diet/lifestyle recommendations.
4. **Implement a symptom checker** that accepts natural language symptom descriptions, extracts relevant symptoms using NLP, matches them against a disease database, and returns the top 3 most likely diseases with confidence scores, urgency levels, and recommended tests.

Secondary Objectives

5. **Enable multilingual support** for 12 Indian languages (Hindi, Bengali, Tamil, Telugu, Marathi, Gujarati, Punjabi, Kannada, Malayalam, Odia, Urdu, English) to make healthcare information accessible to non-English speaking rural populations.
6. **Integrate voice output (Text-to-Speech)** using Google Text-to-Speech (gTTS) so that illiterate and visually impaired users can listen to health information instead of reading it.
7. **Ensure system robustness** through fallback mechanisms including 3-model LLM fallback chains, keyword-based test retrieval when LLM returns insufficient data, and drug-label filtering to maintain content quality.

- Expected outcomes and deliverables.

A fully functional Streamlit web application with 4 interactive tabs

A ChromaDB vector database populated with 500+ MedlinePlus health articles

A JSON database of 1000+ medical tests with detailed information

Integration with OpenFDA API for authoritative drug data

A disease-symptom mapping database for the symptom checker module

Working voice output in 12 languages



Methodology and Results

- **Methods/Technology used**

The project follows a **Retrieval-Augmented Generation (RAG)** architecture combined with modular agent-based design:

1. Retrieval-Augmented Generation (RAG)

- User queries are converted to 384-dimensional embeddings using `SentenceTransformers (all-MiniLM-L6-v2)`
- ChromaDB vector database performs cosine similarity search over MedlinePlus articles
- Top-K relevant documents are retrieved and passed to the LLM as context
- The LLM (Groq API: qwen-27b / gpt-oss-120b / gpt-oss-20b) generates structured JSON responses

2. Modular Agent Architecture

- **DiseaseInfoRetriever:** Handles disease lookup, LLM prompting, JSON parsing, and output cleaning
- **DrugInfoAgent + DrugQA Backend:** Manages drug queries, fuzzy matching, and OpenFDA API integration
- **ReportAnalyzer:** Performs OCR on PDFs/images and LLM-based structured analysis
- **SymptomCheckerAgent:** NLP symptom extraction, disease matching, confidence scoring
- **TranslationAgent:** 12-language medical text translation
- **VoiceOutputAgent:** gTTS-based audio generation

3. Data Processing Pipeline

- **Scraping:** BeautifulSoup + Requests for MedlinePlus medical tests
- **XML Parsing:** English-only topic extraction from MedlinePlus XML dumps
- **OCR:** PyPDF2/pdfplumber for PDFs, PIL for image preprocessing
- **JSON Normalization:** `normalize_name()` function handles brackets, hyphens, spaces for fuzzy matching

- **Tools/Software used**



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

Tool	Version	Purpose
Python	3.10+	Core programming language
Streamlit	1.28+	Web application framework
ChromaDB	Latest	Vector database for RAG
Sentence-Transformers	Latest	Embedding generation
Requests	Latest	HTTP API calls (Groq, OpenFDA)
PyPDF2 / pdfplumber	Latest	PDF text extraction
Pillow (PIL)	Latest	Image processing
gTTS	Latest	Text-to-speech conversion
python-dotenv	Latest	Environment variable management

- **Data collection approach (if applicable)**

Source 1: MedlinePlus (NIH)

- Downloaded XML dumps of health topic articles from medlineplus.gov
- Filtered English-only topics (Spanish articles removed)
- Parsed ~500+ articles into structured format
- Stored in ChromaDB with metadata (title, URL, category)

Source 2: OpenFDA API

- Integrated open.fda.gov API for structured drug label data
- Retrieved: active ingredients, dosage, warnings, side effects, interactions
- Stored locally for fast lookup via Drug QA Backend

Source 3: Web Scraping (Medical Tests)

- Scraped 1000+ medical tests from MedlinePlus Medical Tests section
- Extracted: test name, description, preparation, procedure, normal values, risks
- Stored as structured JSON database

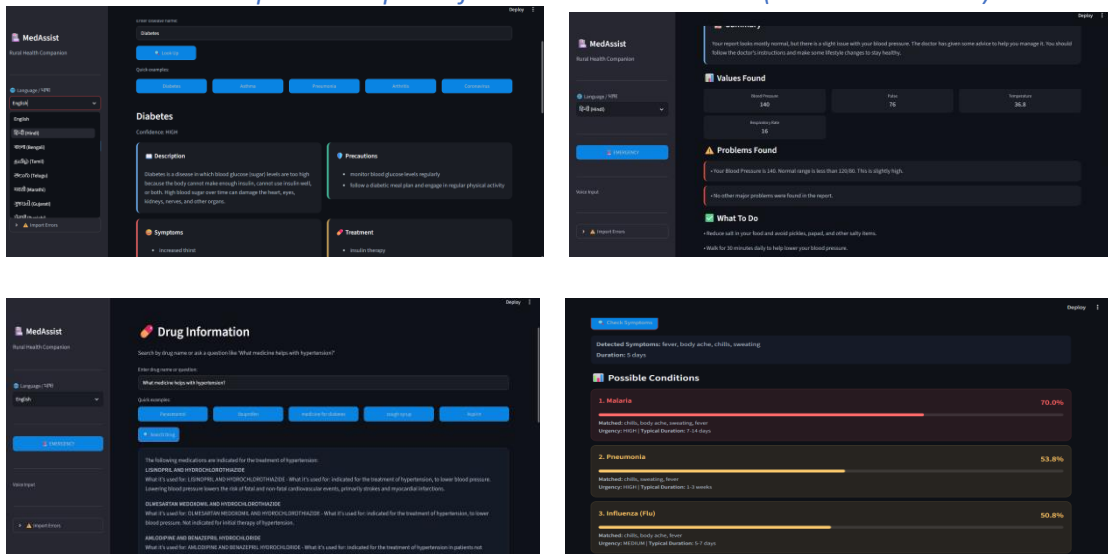
- **Project Architecture (Clearly describe the architecture)**

User (Browser)
↓
Streamlit UI (4 Tabs)
↓
Agent Layer (6 Modules)
↓
[ChromaDB] ↔ [Groq LLM] ↔ [JSON DB] ↔ [OpenFDA API]
↓
Translation + Voice Output

- **Final project working screenshots along with supporting explanation**



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)



Screenshot 1: Disease Information Tab

This screenshot shows the **Disease Information** tab where the user has searched for "Diabetes". The interface displays structured cards containing:

- **Description:** Overview of the disease
- **Symptoms:** List of common symptoms (increased thirst, frequent urination, etc.)
- **Precautions:** Preventive measures
- **Treatment:** Management options
- **Diagnostic Tests:** Clickable buttons (e.g., A1C Test, Fasting Blood Glucose) that open detailed test information
- **Risk Factors** and **When to See a Doctor** The sidebar shows the language selector (12 Indian languages) and voice input option. The dark theme is optimized for low-light rural usage.

Screenshot 2: Medical Report Analyzer Tab

This screenshot demonstrates the **Medical Report Analyzer** tab. A user has uploaded a medical report (PDF/Image), and the system has:

- Extracted text using OCR (PyPDF2/PIL)
- Analyzed the report using Groq LLM
- Displayed **Values Found** (e.g., LDL, HDL, Triglycerides)
- Listed **Problems Found** with severity indicators (e.g., high blood pressure flagged)
- Provided **What To Do** recommendations including diet changes and exercise
- Included a "Listen" button for voice output and options to download or share the analysis



Screenshot 3: Drug Information Tab

This screenshot shows the **Drug Information** tab where the user has searched for a medicine. The system displays:

- **Drug Name** and generic alternatives
- **Indications:** What the medicine treats
- **Dosage:** Recommended intake instructions
- **Side Effects:** Possible adverse reactions
- **Warnings:** Contraindications and precautions
- **Drug Interactions:** Other medicines to avoid The fuzzy matching engine resolves brand names to generic names and handles spelling errors automatically.

Screenshot 4: Symptom Checker Tab

This screenshot displays the **Symptom Checker** tab. The user entered symptoms (fever, body ache, chills, sweating), and the system returned:

- **Detected Symptoms** with parsed duration
- **Top 3 Possible Conditions** with color-coded confidence bars:
 - **Malaria** (79.0% — ☐ High Confidence)
 - **Pneumonia** (53.8% — ☐ Medium Confidence)
 - **Influenza (Flu)** (50.0% — ☐ Medium Confidence)
- Each prediction includes matched symptoms, urgency level, typical duration, and **recommended diagnostic tests** (clickable)
- Emergency detection automatically triggers if critical symptoms are present
- **Project GitHub Link**

<https://github.com/sure-trust/DIVYA-GOYAL-g39-ai-ml/tree/main/Final%20capstone%20project/MedAssist>



- Document every team member's new learnings (in terms of technology, management, etc)

Technology Learnings

Streamlit & Web Development

- Built interactive web applications using Streamlit with multi-tab interfaces and sidebar controls
- Managed session states and resolved widget state conflicts using alternate key patterns

Vector Databases & RAG Architecture

- Implemented ChromaDB for vector storage and cosine similarity search
- Used SentenceTransformers (all-MiniLM-L6-v2) for 384-dimensional text embeddings
- Built the complete RAG pipeline: query → embedding → retrieval → LLM → structured JSON

LLM Integration & API Management

- Integrated Groq API with 3-model fallback chain (qwen-27b → gpt-oss-120b → gpt-oss-20b)
- Developed robust JSON parsing to handle markdown blocks, comments, and trailing commas
- Handled API rate limits, authentication errors, and service unavailability

Data Collection & Processing

- Scraped 1000+ medical tests from MedlinePlus using BeautifulSoup
- Parsed XML dumps to extract ~500 English health topics for ChromaDB
- Integrated OpenFDA API for structured drug data
- Implemented fuzzy string matching and text normalization for test name variations

OCR, Voice & Multilingual Systems

- Extracted text from PDFs (PyPDF2/pdfplumber) and images (PIL) for report analysis
- Integrated gTTS for voice output in 12 Indian languages
- Built a custom Translation Agent supporting Hindi, Bengali, Tamil, Telugu, and 8 more languages

NLP & Symptom Extraction

- Implemented NLP-based symptom extraction from natural language input
- Built disease-symptom matching with confidence scoring and urgency classification

Problem-Solving Learnings

- Learned to design modular architecture with clear data flows and fallback mechanisms



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

- Understood the importance of data quality — implemented drug-label filtering to remove FDA medication guides from disease search results
- Managed environment variables securely and handled network failures gracefully
- Designed for accessibility: voice output and multilingual support for rural, non-English speaking users

- **Document every team member's experience. (overall experience)**

Building MedAssist solo was challenging yet deeply rewarding. The toughest problems were LLM JSON parsing failures, Streamlit widget conflicts, and ensuring accurate drug-label filtering. Each required creative solutions like 3-model fallback chains and alternate session state keys.

The most satisfying part was creating something that addresses a real social need — providing 600+ million rural Indians with instant health information in their own language. The voice feature felt especially impactful as it helps illiterate users access medical guidance.

This project strengthened skills in Python, vector databases, LLM integration, web scraping, and full-stack development — while reinforcing that production-ready AI needs robust error handling, data quality checks, and thoughtful UX design, not just API calls.



Conclusion and Future Scope

- **Recap objectives and achievements**

MedAssist was developed with the objective of providing accessible, reliable, and multilingual healthcare information to rural and underserved populations. All primary objectives have been successfully achieved.

The Disease Information Lookup module was completed with a 4-tab Streamlit application featuring structured disease cards, clickable diagnostic tests, and ChromaDB RAG integration. The Drug Information Query module includes fuzzy brand-to-generic name matching, OpenFDA API integration, and complete display of dosage, side effects, and drug interactions. The Medical Report Analyzer successfully handles PDF and image uploads through OCR, with LLM-based summarization, abnormal value detection, and diet recommendations. The Symptom Checker uses NLP symptom extraction and returns Top-3 disease predictions with confidence scoring and urgency classification. Multilingual support covers 12 Indian languages with working translation across all tabs. Voice output was integrated using gTTS for audio playback of all medical content.

The system handles over 500 MedlinePlus health articles, 1000+ medical tests, and includes 3-model LLM fallback chains for reliability. The drug-label filtering and fuzzy test matching innovations ensure high-quality, relevant output for every query.

- **Future scope of this project**

Short-Term :

Offline Mode — Package ChromaDB with a local lightweight LLM such as Llama 2 7B for zero-internet rural usage where connectivity is unavailable.

WhatsApp Bot — Deploy the symptom checker and disease lookup as a conversational WhatsApp bot for feature-phone users who do not have smartphones.

Offline TTS — Replace gTTS with local Indic TTS engines to eliminate internet dependency for voice output in rural areas.

Medium-Term:



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)

Medical Image Analysis — Add computer vision capabilities for skin rash detection, eye redness analysis, and wound assessment using CNN models.

Persistent Health Records — Enable users to store and track health history, vitals, and report trends over time for chronic condition monitoring.

Community Health Dashboard — Aggregate anonymized symptom data to detect disease outbreak patterns at the village or block level for public health officials.

Long-Term Vision (1 to 2 years):

Telemedicine Integration — Direct video consultation feature with verified local doctors and specialists for remote diagnosis.

Wearable Device Support — Connect with low-cost health wearables such as pulse oximeters and glucometers for real-time vitals monitoring and automated alerts.

AI Health Companion — Upgrade to a conversational AI that maintains context across sessions and handles follow-up queries naturally.



Innovation & Entrepreneurship Hub for Educated Rural Youth (SURE Trust – IERY)